

SLAler-APP — Issue 草稿（3 条）

以下每一节可以独立开一个 issue，直接复制粘贴。行号基于 `main` 分支当前 HEAD。

Issue 1

标题： 设置页修改「第一教学周周一」后，课表页周次不刷新

Labels: `bug`

现象

路径 A（手动设置）

1. 打开课表页，记下当前显示的周次（`WeekScheduleScreen.kt:77` 的标题）
2. 切到设置页，修改「第一教学周周一」或用「本周是第 N 周」快捷设置
3. 切回课表页 —— 周次仍是修改前的值

同时 `state.anchored`（`WeekScheduleScreen.kt:110`）也不更新，所以那条「尚未设置开学周」的提示在设置完之后也不会消失。

路径 B（自动发现，影响更大）

`setSemesterAnchor` 不止设置页会写，正常刷新的学期发现流程也会写：

- `data/schedule/ScheduleRepositoryImpl.kt:327`
- `data/sis/SisRemoteDataSource.kt:353`

也就是说 app 自己从教务系统探测出了第一教学周并落了盘，课表页照样显示旧值或「未设置」。

存续时间

项目没有用 `NavHost`，`hiltViewModel()`（`WeekScheduleScreen.kt:69`）落在 `Activity` 作用域；`manifest` 里又声明了

`configChanges="orientation|screenSize|..."`，`Activity` 连旋转都不重建。所以这个陈旧值会一直保持到进程被杀，切 tab、旋转屏幕都清不掉。

原因

`ScheduleViewModel` 的 `anchor` 是 `init` 里一次性读进来的：

```
// feature/schedule/ScheduleViewModel.kt:115
private val anchor = MutableStateFlow<Pair<LocalDate?, Boolean>>(null to false)

init {
    viewModelScope.launch {
        anchor.value = sessionStore.semesterAnchor() // :119 只读这一次
        sessionManager.hydrate()
    }
}
```

anchor 之后再无写入，而 state 的 combine (:141) 依赖它算 Semester(firstWeekMonday = anchorDate) 和 weekIndex。

根因是 SessionStore 对 anchor 只提供了一次性读：

```
// core/session/SessionStore.kt:123
suspend fun semesterAnchor(): Pair<LocalDate?, Boolean> { ... }
```

值得说明的是这不是设计方向的问题。SessionStore 里 snapshot (:70)、baseUrls (:179)、reminderConfig (:183)、appLanguage (:130)、appTheme (:140)、attendanceGoalMinutes (:222) 全都是 Flow，惯例已经立好了。只是 semesterAnchor / studentIdHint / semesterCodes / integrationMode 这四个 key 停留在 suspend 一次性读，没跟上。

建议

在 SessionStore 加一个 Flow：

```
val semesterAnchorFlow: Flow<Pair<LocalDate?, Boolean>> =
    context.sessionDataStore.data.map { prefs ->
        prefs[Keys.FIRST_WEEK_MONDAY]?.let(LocalDate::ofEpochDay) to
            (prefs[Keys.FIRST_WEEK_ANCHOR_CONFIRMED] ?: false)
    }
```

ScheduleViewModel 直接 combine 它，删掉本地的 anchor MutableStateFlow 和 init 里那次读取。SettingsViewModel 同理（见 Issue 2）。

Issue 2

标题：会话/偏好状态存在双真相源，靠手动同步保持一致

Labels: refactor, architecture

背景

数据层（课表、考勤）的 SSOT 做得很干净：Room 是唯一真相源，`observe*` 不触网，`refresh()` 只在完整校验通过后原子替换，失败不清缓存。`ScheduleRepositoryImpl` 头部注释描述的闭环和实现是一致的。

问题集中在会话和偏好这条线：`DataStore` 明明能出 `Flow`，但被当成一次性读的 KV 在用，于是每处都各自缓了一份，再靠手动调用同步。下面三处是同一个病根。

2.1 `SessionManager` 与 `SessionStore` 各存一份

```
// core/session/SessionManager.kt:41
private val _state = MutableStateFlow(SessionSnapshot())
val state: StateFlow<SessionSnapshot> = _state.asStateFlow()

// :45
suspend fun hydrate() {
    val persisted = store.current()
    _state.value = persisted
}
```

而 `SessionStore.snapshot` (`SessionStore.kt:70`) 已经是 `Flow<SessionSnapshot>` 了。两份同样的事实，靠 `hydrate()` 手工对齐。

`hydrate()` 的调用点也不太对——是 `ScheduleViewModel.init` (`ScheduleViewModel.kt:120`) 在负责唤醒这个单例加载自己。课表 `ViewModel` 不该承担这个职责。

代偿痕迹：因为两份状态会互相滞后，`WeekUiState.needsLogin` 不得不同时 OR 两个来源：

```
// ScheduleViewModel.kt:53
val needsLogin: Boolean
get() = sisState == SessionState.EXPIRED ||
    sisState == SessionState.NEEDS_LOGIN ||
    lastRefresh is RefreshResult.SessionExpired ||
    (lastRefresh is RefreshResult.ServerError && lastRefresh.code == 901)
```

原注释写明了「持久化状态机可能滞后，`probe` 也可能滞后」。这是在给双真相源打补丁，不是在解决它。

建议：`_state` 改为从 `store.snapshot` 派生（`stateIn(appScope, SharingStarted.Eagerly, SessionSnapshot())`），写只走 `DataStore`，删掉 `hydrate()` 和 `ViewModel` 里的调用点。

注意 `set()` 里 `ERROR` 态刻意不落盘（“transient observation, not a durable fact”，:53-56），这个设计是对的，改造时需要保留——可以在 `store` 派生流之上叠一层内存 `overlay`，而

不是整个复制一份。

2.2 SettingsViewModel 写完手动回读

```
// feature/settings/SettingsViewModel.kt:61
private val extras = MutableStateFlow(Extras())

// :126
fun refreshExtras() {
    viewModelScope.launch {
        val (anchor, confirmed) = sessionStore.semesterAnchor()
        extras.value = Extras(firstWeekMonday = anchor, anchorConfirmed = confirmed,
                                studentIdHint = sessionStore.studentIdHint())
    }
}
```

setFirstWeekMonday() (:159) 和 setStudentIdHint() (:181) 都是「写 store → 手动 refreshExtras()」。UDF 的环没闭合，正确形态是写入后由 store 的 Flow 自己推回来。

改成 Flow 之后 extras 和 refreshExtras() 可以整个删掉。

2.3 顺带：一处空操作

```
// SettingsViewModel.kt:224
fun setSisBaseUrl(value: String) {
    viewModelScope.launch {
        sessionStore.setBaseUrl(SchoolSystem.SIS, value.trim())
        extras.value = extras.value // ← 自赋值
    }
}
```

两层问题：StateFlow 对相等值去重，Extras 是 data class，这行不会触发任何发射，是个 no-op；

而且它本来就不需要 —— urlProvider.urls 已经是 Flow（AppUrlProvider.kt:50，源头就是 sessionStore.baseUrls），

setBaseUrl 写完会自动推回 core combine。旁边的 setStuBaseUrl() 没有这行，行为反而是对的。建议直接删。

2.4 小项：轮询而非流

```
// SessionManager.kt
suspend fun awaitAuthenticated(system: SchoolSystem, timeoutMillis: Long = 5_000): Boolean {
    val deadline = System.currentTimeMillis() + timeoutMillis
```

```

while (System.currentTimeMillis() < deadline) {
    if (stateOf(system) == SessionState.AUTHENTICATED) return true
    delay(150)
}
...
}

```

可以直接写成：

```

withTimeoutOrNull(timeoutMillis) {
    state.first { it.stateOf(system) == SessionState.AUTHENTICATED }
} != null

```

功能等价，但少一层轮询，也和项目其余部分的流式风格一致。

建议顺序

2.1 → 2.2 → 顺手清掉 2.3 / 2.4。2.1 改完之后 Issue 1 和 2.2 的改法会变得很自然。

Issue 3

标题： CampusRoot 的 overlay 未使用 rememberSaveable，进程重建后丢失

Labels: bug, good first issue

严重性低，可以放在最后。

现象

不会在旋转屏幕时复现 —— manifest 声明了

configChanges="orientation|screenSize|screenLayout|keyboardHidden|uiMode|density|Activity 不重建，Composition 不销毁，remember 在配置变更中是存活的。

只在**进程被系统回收后重建**时复现（后台停留过久，或开发者选项打开「不保留活动」）：

恢复时底层 tab 会正确还原，但正在显示的全屏浮层（登录 WebView、诊断页、API Provider 页）消失。

登录流程停在后台再回来就要重走。

原因

```

// navigation/CampusRoot.kt:41
var selectedTab by rememberSaveable { mutableStateOf(Tab.HOME) }
var overlay by remember { mutableStateOf<Overlay?>(null) } // ←
:42

```

`selectedTab` 用了 `rememberSaveable`，紧邻的 `overlay` 用的是 `remember`。两行的存活语义不一致，恢复后会得到一个「tab 还在、浮层没了」的半状态。

建议

给 `Overlay` 加 `@Parcelize`（`Overlay.Web` 携带 `url / title / system / isLoginFlow / captureEnabled`，都是可序列化的），`overlay` 改用 `rememberSaveable`。不想引 `kotlin-parcelize` 的话手写一个 `Saver` 也可以。